PRACTICAL 10-A

Object Oriented Programming

AIM:

To implement and understand the concepts specified in Practical 10 of the CEUE203 Object Oriented Programming practical list, using Java. The work covers Part A: Thread Pool, Producer-Consumer, Deadlock Demo and Part B: TransactionProcessor and batch operations.

CODE SCREENSHOT:

DeadlockDemo.java

Part A > Practical 10-A > DeadlockDemo.java > DeadlockDemo > main(String[])

```java
public class DeadlockDemo {
    Run | Debug
    public static void main(String[] args) {
        Object a = new Object(), b = new Object();
        Thread t1 = new Thread(() -> {
            synchronized (a) {
                sleep();
                synchronized (b) {
                    System.out.println("t1 acquired both");
                }
            }
        });
        Thread t2 = new Thread(() -> {
            synchronized (b) {
                sleep();
                synchronized (a) {
                    System.out.println("t2 acquired both");
                }
            }
        });
        t1.start();
        t2.start();
        System.out.println("Demo may deadlock; stop manually if both threads block.");
    }

    static void sleep() {
        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

Code Screenshot 1

ProducerConsumer.java

Part A > Practical 10-A > ProducerConsumer.java > ...

```java
import java.util.concurrent.*;

public class ProducerConsumer {
    Run | Debug
    public static void main(String[] args) throws Exception {
        BlockingQueue<Integer> buffer = new ArrayBlockingQueue<>(3);
        Thread producer = new Thread(() -> {
            try {
                for (int i = 1; i <= 5; i++) {
                    buffer.put(i);
                    System.out.println("Produced " + i);
                }
                buffer.put(-1);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });
        Thread consumer = new Thread(() -> {
            try {
                while (true) {
                    int n = buffer.take();
                    if (n == -1)
                        break;
                    System.out.println("Consumed " + n);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });
        producer.start();
        consumer.start();
        producer.join();
        consumer.join();
    }
}

```

Code Screenshot 2

ThreadPoolRunner.java

Part A > Practical 10-A > ThreadPoolRunner.java > ...

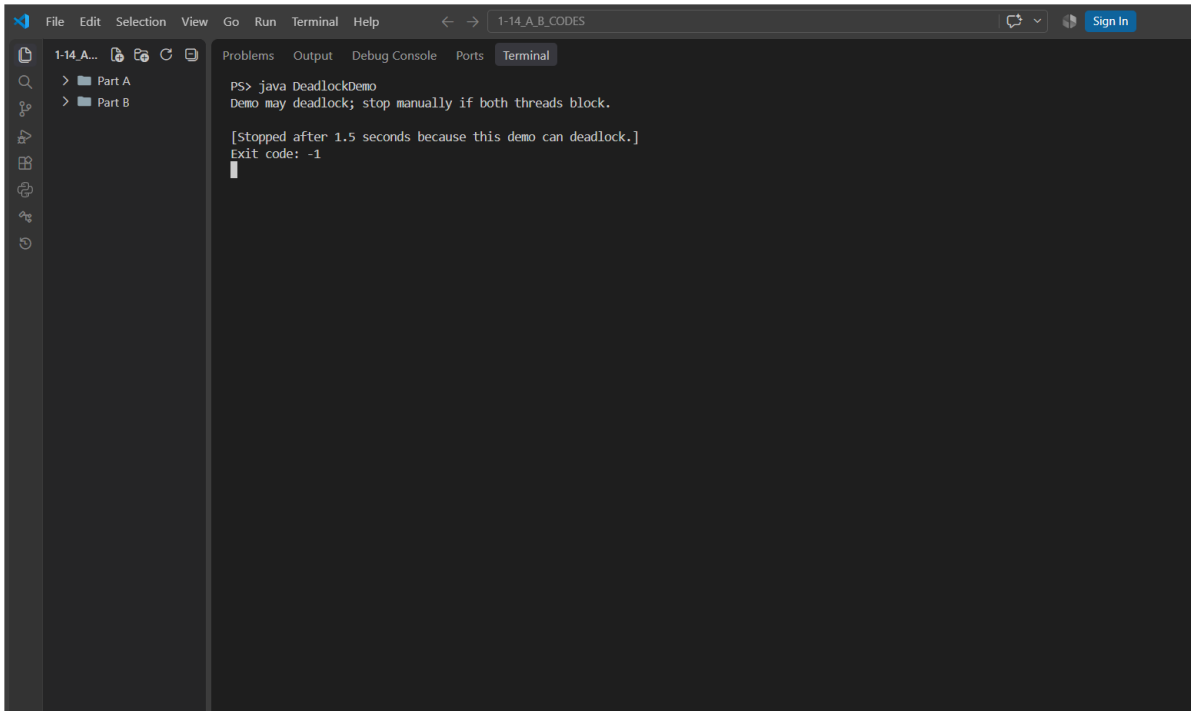
```java
import java.util.concurrent.*;

public class ThreadPoolRunner {
    Run | Debug
    public static void main(String[] args) throws InterruptedException {
        ExecutorService pool = Executors.newFixedThreadPool(3);
        for (int i = 1; i <= 10; i++) {
            final int n = i;
            pool.submit(() -> System.out.println("Task " + n + " on " + Thread.currentThread().getName()));
        }
        pool.shutdown();
        pool.awaitTermination(1, TimeUnit.MINUTES);
    }
}

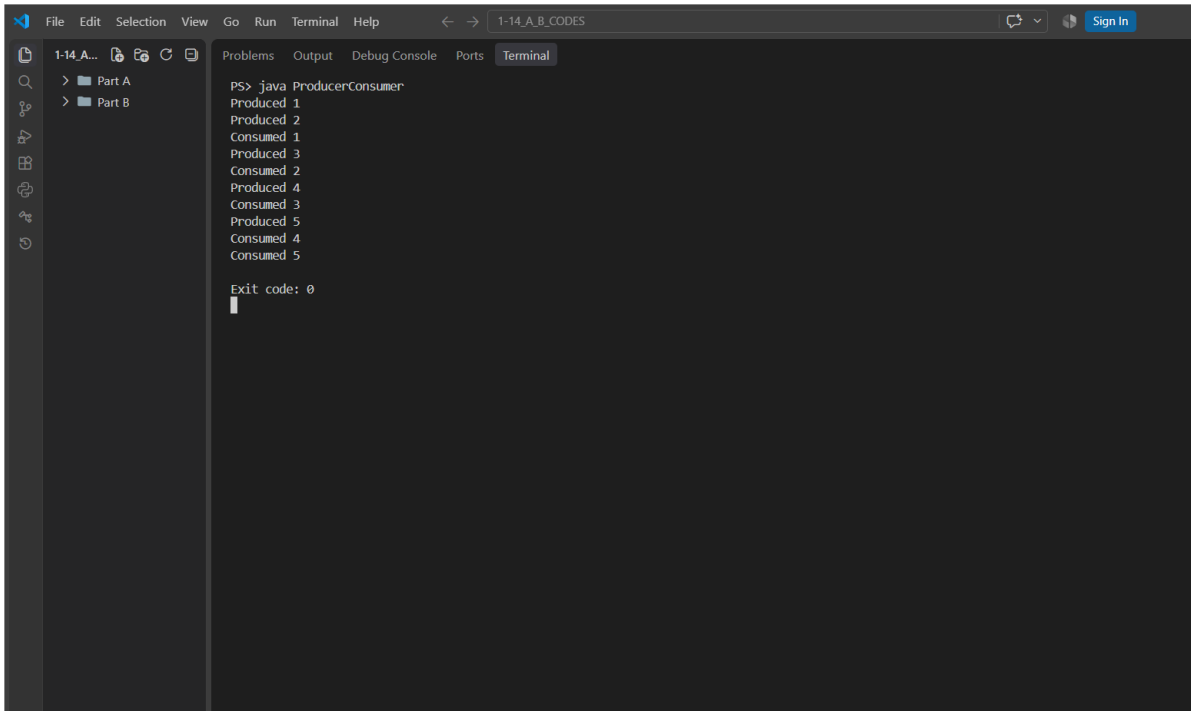
```

Code Screenshot 3

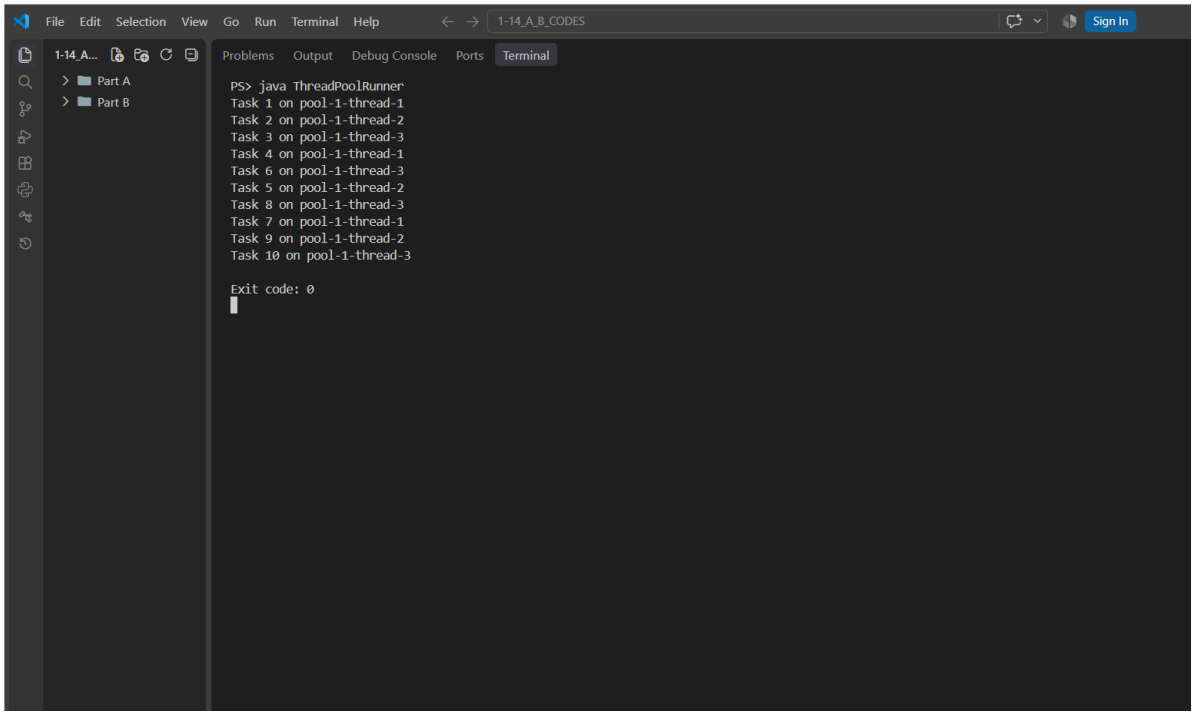
OUTPUT SCREENSHOT:



Output Screenshot 1



Output Screenshot 2



Output Screenshot 3

LEARNING OUTCOMES:

Use executor services and producer-consumer coordination to manage concurrent tasks.